

CryptoQuant Terminal Arquitectura v4.0

Revision critica de las decisiones estructurales del roadmap. Evaluado desde 4 perspectivas: Quant Architect, Portfolio Manager, Risk Manager y CTO. Este documento reemplaza todos los anteriores y define la arquitectura definitiva para implementacion.

4 de junio de 2026

Version 4.0 — Final

Clasificacion: Ejecutivo

Contenido

1. Declaracion de Principios	4
2. Revision 1: Sistema de Pesos	4
2.1 La propuesta actual	4
2.2 Por que los pesos fijos son incorrectos	4
2.3 La alternativa: Veto Jerarquico + Confidence-Weighted Scoring	5
2.4 Veredicto	6
3. Revision 2: Validation Grade	6
3.1 La propuesta actual	6
3.2 Por que los grados academicos son una simplificacion peligrosa	6
3.3 La alternativa: Action-First Classification	7
3.4 Veredicto	8
4. Revision 3: Capital Allocation	8
4.1 Estado actual: 16 metodos, 0 consumidores	8
4.2 Cuales implementar para V1	8
4.3 Que eliminar o retrasar	9
4.4 Veredicto	10
5. Revision 4: Decision Audit Engine	10
5.1 El concepto	10
5.2 Deberia existir desde el principio?	10
5.3 Arquitectura minima viable	11

5.4 Posicion en el roadmap	11
5.5 Veredicto	11
6. Revision 5: Evolution Tree	12
6.1 Estado actual: visual sin valor operativo	12
6.2 Conversion a fuente de candidatos para el SDE	12
6.3 Veredicto	12
7. Revision 6: El Cuello de Botella Real	13
7.1 Veredicto	13
8. Revision 7: Que NO Construir	14
8.1 Eliminar del codigo	14
8.2 Retrasar hasta V2+	14
8.3 Simplificar	15
8.4 Sobreingenieria detectada	16
8.5 Veredicto	16
9. Arquitectura Final Consolidada	16
9.1 Pipeline completo	16
9.2 SDE: Logica de decision completa	17
9.3 Modelo de datos consolidado	18
10. Roadmap Revisado	18
10.1 Sprint 0: Foundation Fixes [3-4 dias]	18
10.2 Sprint 1: Strategy Decision Engine [2-3 semanas]	19
10.3 Sprint 2: Kill Switches + Risk Budget [1-2 semanas]	19

10.4 Sprint 3: Portfolio Monitor + Volatility Regime [1-2 semanas]	20
--	----

10.5 Sprint 4+ (V2): Mejoras institucionales [post-validacion]	20
--	----

11. Decisiones de Diseno Fundamentales	21
--	----

1. Declaracion de Principios

Este documento REEMPLAZA todos los documentos anteriores: STRATEGIC_PLAN.md, STRATEGIC_ROADMAP_V2.md, y los PDFs de analisis critico v1/v2/v3. A partir de este momento, este es el UNICO documento de referencia arquitectonica del proyecto.

Este documento NO es:

Una nueva auditoria completa. No es una lista infinita de ideas. No es un documento teorico. No propone nuevos modulos ni nuevos dashboards.

Este documento SI es:

Una revision de las decisiones estructurales mas importantes del roadmap existente. Cada decision se evalua desde 4 perspectivas simultaneas: Quant Architect, Portfolio Manager, Risk Manager y CTO de una firma cuantitativa. Si una decision es correcta, se confirma. Si es incorrecta, se sustituye con una alternativa concreta. Si algo critico falta, se anade. Si algo es sobreingenieria, se elimina.

El objetivo es cerrar la arquitectura y comenzar a construir.

Cada seccion de este documento contiene una decision final, no una propuesta. Las secciones con veredicto son ejecutivas: definen exactamente que se implementa, que se retrasa, y que se elimina.

2. Revision 1: Sistema de Pesos

2.1 La propuesta actual

El roadmap v2.0 proponia pesos fijos para el Strategy Decision Engine: Monte Carlo 30%, Walk Forward 25%, Backtest 25%, Operability 10%, Regime 10%. La idea intuitiva es atractiva: cada modulo contribuye una fraccion conocida al score final, y se ponderan por importancia relativa.

2.2 Por que los pesos fijos son incorrectos

Los pesos fijos tienen tres problemas fundamentales que los hacen inaceptables para un sistema cuantitativo de trading:

Problema 1: Ignoran la confianza del modulo emisor. Un Monte Carlo con 50 simulaciones tiene menos confianza estadistica que uno con 10,000 simulaciones. Sin embargo, con pesos fijos, ambos contribuyen el mismo 30% al score final. Esto es equivalente a darle el mismo peso al testimonio de un testigo presencial y al de alguien que escuchó un rumor.

Problema 2: Son incompatibles con cambios de regimen. En un mercado lateral de baja volatilidad, el backtest historico es mas predictivo que el Monte Carlo. En un mercado con volatilidad extrema (flash crash, evento de liquidez), el Monte Carlo y los kill switches son mas importantes que el backtest. Pesos fijos no pueden adaptarse a esta realidad.

Problema 3: Crean una falsa sensacion de precision. Un score de 73.5/100 sugiere precision que no existe. La diferencia entre 73.5 y 72.8 no es significativa dada la incertidumbre en los inputs. Los pesos fijos animan a tratar este numero como si tuviera mas informacion de la que realmente contiene.

2.3 La alternativa: Veto Jerarquico + Confidence-Weighted Scoring

La arquitectura correcta para el SDE es un sistema de dos capas: vetos absolutos primero, scoring ponderado por confianza despues. Esto no es teoria; es la estructura exacta que implementaremos.

Capa 1: Vetos Absolutos (hard gates)

Los vetos son binarios: pasas o no pasas. No hay pesos, no hay scores parciales. Si cualquier veto falla, la estrategia es REJECT. No importa que los demas modulos den scores perfectos. Esta capa se ejecuta PRIMERO y es obligatoria para TODAS las estrategias, sin excepcion.

Veto	Condicion	Perfil Conservador	Perfil Moderado	Perfil Agresivo
Risk of Ruin	RoR > threshold	< 1%	< 3%	< 5%
Total Trades	Trades < min	< 50	< 40	< 30
Max Drawdown	DD > limit	> 40%	> 50%	> 60%
Walk-Forward Eff.	WFE < min	< 40%	< 30%	< 20%
Win Rate + Payoff	WR < 35% Y Payoff < 2.5:1	Ambos	Ambos	Ambos

Tabla 1. Vetos absolutos por perfil de riesgo

Capa 2: Confidence-Weighted Scoring

Solo si la estrategia pasa TODOS los vetos, se calcula un score compuesto. Pero los pesos no son fijos: cada modulo contribuye en proporcion a su propia confianza en el output que esta produciendo. Esto es fundamentalmente diferente de pesos fijos y resuelve los tres problemas anteriores.

La formula es:

```
CompositeScore = SUM(module_score[i] * module_confidence[i]) /
SUM(module_confidence[i]) Donde: module_score[i] = output normalizado del modulo
i (0-100) module_confidence[i] = confianza del modulo i en su propio output (0-1)
Ejemplos de confidence: MC con 10000 sims: confidence = 0.95 MC con 100 sims:
```

confidence = 0.40 WF con 8 ventanas: confidence = 0.90 WF con 2 ventanas:
confidence = 0.30 Backtest 2 años: confidence = 0.85 Backtest 30 días:
confidence = 0.20

Esta arquitectura tiene tres propiedades críticas que los pesos fijos no tienen:

Primero, se adapta automáticamente a la calidad de los datos disponibles. Si el Monte Carlo se ejecutó con pocas simulaciones, su confianza baja y su influencia en el score compuesto disminuye proporcionalmente. No requiere intervención manual ni cambios de configuración.

Segundo, es naturalmente robusta frente a módulos con outputs ruidosos. Un módulo que produce scores con alta varianza entre ejecuciones debería tener baja confianza, y por lo tanto bajo peso efectivo. Esto desincentiva la sobreoptimización de un solo módulo.

Tercero, es transparente y auditable. Cuando el SDE produce una decisión, puede reportar exactamente cuánto peso efectivo tuvo cada módulo y por qué. "El Monte Carlo contribuyó 42% del score porque su confianza era 0.95, mientras que el Walk-Forward solo contribuyó 15% porque solo tenía 2 ventanas". Esta transparencia es esencial para la confianza del usuario en el sistema.

2.4 Veredicto

VEREDICTO: Pesos fijos ELIMINADOS. Se implementa Veto Jerárquico + Confidence-Weighted Scoring. Los vetos son absolutos y por perfil de riesgo. Los scores son ponderados por confianza del módulo emisor. No hay pesos hardcodeados en ningún lugar del sistema.

3. Revision 2: Validation Grade

3.1 La propuesta actual

El roadmap v2.0 proponía un sistema de grados A+/A/B/C/D/F, similar al sistema académico estadounidense. La intención era comunicar rápidamente la calidad de una estrategia con una etiqueta simple.

3.2 Por qué los grados académicos son una simplificación peligrosa

Los grados tipo A/B/C/D/F tienen tres problemas que los hacen inadecuados para un sistema de trading:

Problema 1: Son teatrales, no operativos. Un "Grade B" no te dice qué hacer. Significa "bastante bueno" en términos vagos, pero no especifica: debes asignar capital? Cuánto? Debes paper-tradeear primero? La etiqueta comunica una sensación, no una acción. En un sistema que aspira a ser semi-autónomo, la salida del SDE debe ser directamente accionable.

Problema 2: Implican falsa precision. La frontera entre B y C es arbitraria. Un score de 59.9 es C, un score de 60.0 es B. Pero la diferencia real es ruido estadistico. Los grados academicos sugieren que existe una distincion significativa donde solo hay ruido.

Problema 3: No capturan la dimension de accion. El sistema necesita distinguir entre "esta estrategia es excelente pero ya tiene suficiente capital asignado" (HOLD) y "esta estrategia es excelente y merece mas capital" (INCREASE). Los grados academicos no tienen esta dimension.

3.3 La alternativa: Action-First Classification

El reemplazo es un sistema donde la clasificacion ES la accion. Cada estrategia recibe una Action Tag que define exactamente que hacer con ella, acompañada de un Confidence Score numerico que determina el tamano de posicion dentro de esa accion.

Action Tag	Significado	Accion de Capital	Rango de Confidence
TRADE	Estrategia validada, lista para capital real	Asignar capital segun allocation method	70-100
WATCH	Estrategia prometedora, necesita mas datos	Paper trading solo, sin capital real	40-69
HOLD	Estrategia activa, mantener posicion actual	Mantener allocation actual, no aumentar	50-69
RETRAIN	Estrategia degradada, necesita ajuste	Reducir capital al 50%, reoptimizar	25-49
REJECT	Estrategia no valida	Cerrar posiciones, no asignar capital	0-24

Tabla 2. Action-First Classification: cada tag es una accion directa

El Confidence Score (0-100) se calcula a partir del Confidence-Weighted Composite Score definido en la Revision 1. Este score determina el tamano de posicion dentro de la accion:

```
Si Action = TRADE: position_multiplier = confidence / 100 kelly_fraction =  
base_kelly * position_multiplier Si Action = WATCH: Solo paper trading, no  
position sizing Si Action = HOLD: Mantener posicion actual sin cambios Si Action  
= RETRAIN: Reducir a 50% de posicion actual Si Action = REJECT: Cerrar toda  
posicion
```

La interfaz TypeScript del SDE output queda asi:

```
interface SDEDecision { strategyId: string; action: "TRADE" | "WATCH" | "HOLD" |  
"RETRAIN" | "REJECT"; confidence: number; // 0-100 vetoResults: VetoCheck[]; //  
que vetos se evaluaron moduleContributions: { // transparencia total moduleId:  
string; score: number; // 0-100 confidence: number; // 0-1, peso efectivo  
effectiveWeight: number; // peso real en la decision }[]; capitalAction:
```



```
"ALLOCATE" | "HOLD" | "REDUCE" | "EXIT"; targetAllocationPct: number; // % del
portfolio allocationMethod: AllocationMethod; reasoning: string; // explicacion
en lenguaje natural nextReviewDate: Date; }
```

3.4 Veredicto

VEREDICTO: Grados A+/A/B/C/D/F ELIMINADOS. Se implementa Action-First Classification con 5 tags (TRADE/WATCH/HOLD/RETRAIN/REJECT) + Confidence Score numerico (0-100). Cada tag es directamente accionable. El Confidence Score determina position sizing dentro de la accion. No hay etiquetas ambiguas.

4. Revision 3: Capital Allocation

4.1 Estado actual: 16 metodos, 0 consumidores

El CapitalAllocationEngine actual implementa 16 metodos de allocation. Es codigo impresionante: Risk Parity con Spinu 2013, Markowitz con Gauss-Jordan, Kelly fraccional, volatilidad targeting. Pero tiene un problema critico: el Paper Trading Engine no lo usa. La funcion calculatePositionSize() es un simple availableCapital / remainingSlots que ignora completamente los 16 metodos.

Ademas, nadie decide QUE metodo usar y CUANDO. La eleccion es manual y arbitraria. Es como tener un garage lleno de ferraris pero ir caminando al trabajo porque nadie te dijo cual llave usar.

4.2 Cuales implementar para V1

Para una primera version autonomo, necesitas exactamente 3 metodos. No 5, no 8, no 16. Tres. Cada uno con un proposito claro y una regla de seleccion definida:

Metodo	Funcion	Cuando se usa	Default
Half-Kelly	Position sizing optimo para crecimiento a largo plazo	Default para estrategias individuales con Action=TRADE	SI
Risk Parity	Allocacion portfolio-level entre estrategias no correlacionadas	Cuando 2+ estrategias activas simultaneamente	NO
Volatility Targeting	Reducir exposicion cuando volatilidad sube	Cuando volatilidad del regimen > percentil 75	NO

Tabla 3. Los 3 metodos de Capital Allocation para V1

Half-Kelly como default es la decision correcta porque: es el unico metodo que maximiza crecimiento a largo plazo con riesgo controlado (demostrado por Kelly 1956 y confirmado por Thorp); usar la mitad del Kelly full es el estandar de la industria porque reduce la varianza del crecimiento a cambio de una pequena reduccion en retorno esperado; y es robusto frente a errores de estimacion en los parametros de entrada (win rate, payoff ratio), algo critico en crypto donde los datos son ruidosos.

Risk Parity para portfolio-level porque: es superior a equal-weight ya que ajusta por volatilidad (una estrategia con volatilidad 5% no merece la misma allocation que una con volatilidad 40%); la implementacion Spinu 2013 ya existe en el codigo actual y es correcta; y es la herramienta adecuada cuando el SDE gestiona multiples estrategias simultaneamente.

Volatility Targeting como mecanismo de defensa porque: es la respuesta automatica a regimen de alta volatilidad (cuando el mercado se vuelve erratico, reduces exposure); se implementa con una sola formula: $\text{target_vol} / \text{realized_vol}$; y es el complemento natural del regime detector (simplificado a volatilidad para V1).

4.3 Que eliminar o retrasar

Metodo	Accion	Razon
FIXED_AMOUNT	ELIMINAR	No escala con capital, inutil para portfolio
FIXED_RATIO	ELIMINAR	Demasiado conservador, no adaptativo
FIXED_FRACTIONAL	RETRASAR	Subconjunto de Kelly, redundante para V1
MIN_VARIANCE	ELIMINAR	Subset de Markowitz, mismos problemas de estimacion
RL_ALLOCATION	ELIMINAR	Q-table es un juguete sin simulacion real
META_ALLOCATION	ELIMINAR	Necesita datos que no tenemos todavia
SCORE_BASED	RETRASAR	Redundante con Confidence Score del SDE
REGIME_BASED	RETRASAR	Se reemplaza por Volatility Targeting para V1
ADAPTIVE	RETRASAR	Complejidad innecesaria para V1
CUSTOM_COMPOSITE	RETRASAR	Solo util cuando hay 4+ metodos activos
MEAN_VARIANCE	RETRASAR	Demasiado sensible a errores de estimacion en crypto

EQUAL_WEIGHT	RETRASAR	Risk Parity es superior, este es fallback
MAX_DRAWDOWN N_CONTROL	RETRASAR	Se reemplaza por kill switches para V1

Tabla 4. Metodos de Capital Allocation: eliminar o retrasar

4.4 Veredicto

VEREDICTO: 16 metodos reducidos a 3 para V1. Half-Kelly (default, position sizing), Risk Parity (portfolio allocation), Volatility Targeting (regimen defensivo). 4 metodos ELIMINADOS del codigo. 9 metodos RETRASADOS a V2+. El Paper Trading Engine se conecta al CapitalAllocationEngine usando estos 3 metodos.

5. Revision 4: Decision Audit Engine

5.1 El concepto

El Decision Audit Engine es un mecanismo que registra las predicciones del SDE y las compara con los resultados reales. Cuando el SDE decide que una estrategia es TRADE con confidence 82%, el audit engine registra esa prediccion. Treinta dias despues, compara: la estrategia realmente rindio como se predijo? Si la estrategia produjo +18% cuando el SDE predijo rendimiento positivo, la decision fue correcta. Si produjo -12%, la decision fue incorrecta. Esta comparacion sistematica es lo que convierte al SDE de un sistema de opinions a un sistema que aprende.

5.2 Deberia existir desde el principio?

Si. Absolutamente. Sin excepcion. El Decision Audit Engine no es un modulo separado que se anade despues; es una propiedad fundamental del SDE que debe existir desde la primera linea de codigo. Sin el, el SDE es un sistema de open-loop: produce decisiones pero nunca verifica si fueron correctas. Es como conducir un auto con los ojos vendados, confiando en que el GPS te dijo que giraras a la derecha hace 10 minutos pero sin poder verificar si llegaste al destino.

Ademas, el Decision Audit Engine es MAS importante que mejorar Monte Carlo o Walk-Forward. La razon es directa: mejorar MC o WF optimiza los INPUTS del SDE, pero el audit engine optimiza el OUTPUT. Puedes tener los mejores inputs del mundo, pero si nunca verificas si las decisiones finales fueron correctas, no sabes si el sistema funciona. El audit engine es el feedback loop que cierra el circuito y permite que el sistema mejore con el tiempo.

5.3 Arquitectura minima viable

La implementacion minima no es un modulo separado. Son dos extensiones al modelo DecisionLog existente en Prisma y un job de reconciliacion:

```
// Extension 1: Campos de prediccion en DecisionLog model DecisionLog { // ...
campos existentes ... // Prediccion al momento de la decision predictedAction
String // TRADE/WATCH/HOLD/RETRAIN/REJECT predictedConfidence Float // 0-100
predictedReturnPct Float? // retorno esperado (nullable) predictedMaxDD Float? //
drawdown maximo esperado // Resultado real (llenado por reconciliation job)
actualReturnPct Float? // retorno real despues de N dias actualMaxDD Float? //
drawdown maximo real actualOutcome String? // PROFIT/LOSS/BREAKEVEN
predictionCorrect Boolean? // la prediccion fue correcta? reconciledAt DateTime?
// cuando se reconcilio reconciliationDays Int? // dias entre decision y
reconciliacion }
```

```
// Extension 2: Reconciliation job (se ejecuta diariamente) async function
reconcileDecisions() { // Buscar decisiones de hace >= 30 dias sin reconciliar
const unreconciled = await prisma.decisionLog.findMany({ where: { reconciledAt:
null, createdAt: { lte: subDays(now(), 30) } } }); for (const decision of
unreconciled) { // Calcular resultado real desde paper trading trades const
actualReturn = calculateActualReturn(decision); const actualDD =
calculateActualMaxDD(decision); // Comparar prediccion vs realidad const
predictionCorrect = evaluatePrediction(decision, actualReturn); await
prisma.decisionLog.update({ where: { id: decision.id }, data: { actualReturnPct:
actualReturn, actualMaxDD: actualDD, predictionCorrect, reconciledAt: new Date(),
reconciliationDays: 30 } }); }
```

Con esta arquitectura minima, el sistema puede responder a la pregunta mas importante que un quant puede hacer: "de las ultimas 100 decisiones del SDE, cuantas fueron correctas?" Si la tasa de acierto es < 50%, el SDE esta funcionando peor que azar y necesita recalibracion. Si es > 70%, el sistema tiene edge real. Sin el audit engine, esta pregunta es imposible de responder.

5.4 Posicion en el roadmap

El Decision Audit Engine NO es un sprint separado. Se implementa como parte del Sprint 1 (Strategy Decision Engine), porque es una propiedad intrinseca del SDE, no un addon. Los campos de prediccion se anaden al mismo tiempo que los campos de MC/WF scores. El reconciliation job se implementa en Sprint 1 como un cron job diario.

5.5 Veredicto

VEREDICTO: Decision Audit Engine es CORE FEATURE del SDE, no un modulo separado. Se implementa en Sprint 1 junto con el SDE. Los campos de prediccion se anaden a DecisionLog. El reconciliation job se ejecuta diariamente. Sin esto, el SDE es open-loop y no puede aprender.

6. Revision 5: Evolution Tree

6.1 Estado actual: visual sin valor operativo

El Evolution Tree actual es puramente visual. Muestra un arbol de estrategias padre-hija con estadísticas de backtest y porcentaje de mejora, pero no tiene ningun mecanismo de promocion automatica. Si una estrategia hija supera a la padre por 40%, el arbol lo muestra con un badge verde, pero nadie hace nada al respecto. El usuario tiene que manualmente activar la hija y desactivar la padre. Para un sistema semi-autonomo, esto es inaceptable.

6.2 Conversion a fuente de candidatos para el SDE

La Evolution Tree debe convertirse en una fuente de candidatos para el SDE. Cuando una estrategia descendiente supera a su ancestro por un umbral definido (ej: improvementPct > 15%), el arbol genera automaticamente un evento que el SDE evalua. El flujo es:

```
1. Evolution Engine produce estrategia hija
2. Si improvementPct > PROMOTION_THRESHOLD (default: 15%):
  a. Generar evento "CANDIDATE_PROMOTION"
  b. SDE evalua la hija: MC + WF + vetos
  c. Si SDE verdict = TRADE:
    - Sugerir INCREASE_CAPITAL para la hija
    - Sugerir PAUSED para la madre
    - Migracion gradual de capital (no swap instantaneo)
  d. Si SDE verdict != TRADE:
    - Registrar que la hija no paso validacion
    - La madre mantiene su status
3. Si improvementPct <= PROMOTION_THRESHOLD:
  - La hija se queda en el arbol como referencia
  - No se envia al SDE
```

No es demasiado pronto para esto? No. La promocion automatica de descendientes es el mecanismo natural que cierra el loop entre evolucion y deployment de capital. Sin el, la evolucion produce estrategias mejores que nadie usa, lo que es optimizacion sin retorno. El SDE ya debe existir para Sprint 1, y la integracion con Evolution Tree es simplemente conectar dos componentes que ya existen.

Sin embargo, la migracion de capital debe ser gradual, no instantanea. Cuando el SDE aprueba una hija para TRADE, no se transfiere todo el capital de la madre a la hija en un solo paso. Se migra en tramos: primero 25% del capital de la madre, luego 50%, y finalmente el 100% restante, con verificacion del SDE en cada paso. Si la hija no rinde como se esperaba durante la migracion, el proceso se detiene automaticamente.

6.3 Veredicto

VEREDICTO: Evolution Tree pasa de visual a fuente de candidatos del SDE. Promocion automatica con umbral de 15% de mejora. El SDE valida antes de promover. Migracion gradual de capital en tramos de 25%/50%/100%. Se implementa en Sprint 2 (despues de que el SDE funcione en Sprint 1).

7. Revision 6: El Cuello de Botella Real

La unica cosa que impide que esta plataforma llegue a operar de forma semi-autonoma es: el feedback loop roto.

El sistema puede backtestear, puede simular Monte Carlo, puede hacer Walk-Forward, puede evolucionar estrategias, puede hacer paper trading. Pero no puede OBSERVAR SUS PROPIOS RESULTADOS Y AJUSTARSE.

Este es un sistema de open-loop: produce decisiones (o informacion que un humano convierte en decisiones), pero nunca verifica si esas decisiones fueron correctas, ni ajusta su comportamiento en base a los resultados. Es como un termostato que solo puede subir la temperatura pero nunca verificar si la temperatura actual es la correcta.

Los tres componentes del feedback loop roto son:

- 1. El Decision Audit Engine no existe.** El sistema no registra predicciones ni las compara con resultados reales. No sabe si sus decisiones fueron correctas o incorrectas. Esto ya se trato en la Revision 4 y la solucion es la misma: campos de prediccion en DecisionLog + reconciliation job diario.
- 2. El feedback-loop-engine.ts existe pero esta desconectado.** El archivo existe en el codigo, pero no hay mecanismo automatico que alimente los resultados de paper trading de vuelta a la optimizacion de parametros. El sistema evoluciona estrategias con backtests historicos pero no aprende de las operaciones en vivo. Es como estudiar para un examen con apuntes viejos pero nunca verificar las respuestas del examen real.
- 3. Los kill switches no son automaticos.** Actualmente, si una estrategia pierde catastroficamente, el usuario tiene que pausarla manualmente. Para un sistema semi-autonomo, esto es inaceptable. Los kill switches deben ser automaticos (portfolio DD > 20% = pausar todo, estrategia DD > 30% = pausar estrategia), como se define en la Revision 7.

Todo lo demas es optimizacion de inputs: mejores simulaciones MC, mejores analisis WF, mejor deteccion de regimen. Pero optimizar inputs sin cerrar el feedback loop es como calibrar un telescopio con mas precision pero nunca mirar a traves de el para verificar si las estrellas estan donde predijiste. El feedback loop es lo que convierte un sistema de informacion en un sistema de aprendizaje.

7.1 Veredicto

VEREDICTO: El cuello de botella es el feedback loop roto. Todo lo demas es secundario. El Decision Audit Engine (Revision 4) es la prioridad numero 1 porque es la primera mitad del loop. Los kill switches automaticos son la segunda mitad. Ambos se implementan en Sprint 1 y Sprint 3 respectivamente.

8. Revision 7: Que NO Construir

Esta es la revision mas importante. Cada decision de "no construir" libera tiempo y reduce complejidad. Cada pieza de sobreingenieria eliminada reduce la superficie de ataque para bugs y la carga cognitiva para el desarrollador.

8.1 Eliminar del codigo

Estos metodos y modulos se eliminan del codigo fuente porque añaden complejidad sin retorno operativo:

Componente	Razon de eliminacion
RL_ALLOCATION (Q-table)	Juguete sin entorno de simulacion. No hay forma de entrenar el Q-table con datos reales. Produce allocations arbitrarias.
META_ALLOCATION	Necesita historial de rendimiento de multiples allocation methods, que no existe. Es una meta-optimizacion sin datos.
FIXED_AMOUNT	No escala con tamano de portfolio. Asignar \$100 por trade funciona con \$1000 pero no con \$100,000.
FIXED_RATIO (Ryan Jones)	Demasiado conservador para crypto. El delta-based scaling reduce position size cuando deberia aumentarse en tendencias fuertes.
MIN_VARIANCE portfolio	Subset de Markowitz con los mismos problemas de estimacion. No añade nada que Risk Parity no cubra mejor.

Tabla 5. Componentes a eliminar del codigo

8.2 Retrasar hasta V2+

Estos componentes son validos pero no son necesarios para que el sistema funcione. Se retrasan hasta que V1 demuestre que el SDE produce decisiones correctas:

Componente	Cuando reactiva r	Condicion para reactivar
Monte Carlo Block Bootstrap	V2	Cuando el MC actual demuestre que el shuffle destruye estructura temporal significativa
MC Stress Scenarios	V2	Cuando el paper trading tenga 100+ trades y el baseline MC funcione
WF Parameter Drift Analysis	V2	Cuando el WF actual demuestre que parameterStability no es suficiente

WF IS vs OOS Equity Curves UI	V2	Cuando el WF tenga UI y los datos sean utiles para el usuario
WF Overfitting Probability	V2	Como metrica adicional, no como veto
Market Regime Detector 3 capas	V2	V1 usa solo volatilidad; anadir tendencia y liquidez cuando V1 funcione
Cross-strategy correlation	V2	Cuando haya 3+ estrategias activas simultaneamente
MEAN_VARIANCE (Markowitz)	V2	Cuando los datos de covarianza sean estables y significativos
ADAPTIVE position sizing	V3	Complejidad innecesaria hasta que el sistema tenga track record
CUSTOM_COMPOSITE allocation	V3	Solo util cuando haya 4+ metodos de allocation activos
Execution Layer (Jupiter)	V3	DESPUES de que paper trading demuestre Sharpe > 1.0 y kill switches esten probados

Tabla 6. Componentes a retrasar hasta V2/V3

8.3 Simplificar

Estos componentes existen pero necesitan simplificacion para ser utiles en V1:

Componente	Estado actual	Simplificacion V1
Validation Grade	A+/A/B/C/D/F	Action-First: TRADE/WATCH/HOLD/RETRAIN/REJECT
Sistema de pesos	Fijos (MC 30%, WF 25%, etc.)	Veto + Confidence-Weighted Scoring
Capital Allocation	16 metodos	3 metodos: Half-Kelly, Risk Parity, Vol Targeting
Regime Detection	3 capas (vol + trend + liquidity)	1 capa: volatilidad unica
Evolution Tree	Visual sin promocion	Fuente de candidatos para SDE con umbral 15%
Kill Switches	Solo manual	Automaticos: portfolio 20%, estrategia 30%, posicion 50%
DecisionLog schema	Sin campos de prediccion	Campos de prediccion + reconciliacion
parameterStability (WF)	Es win rate stability, no drift	Renombrar a winRateStability, anadir drift real en V2

Tabla 7. Simplificaciones para V1

8.4 Sobreingeniería detectada

Tres patrones de sobreingeniería claros se detectaron en el roadmap v2.0:

1. Explosion de metodos de allocation (16 metodos). Es tentador implementar todos los metodos del libro de texto, pero en un sistema que no ha demostrado todavía que puede tomar una sola decisión correcta, tener 16 formas de asignar capital es indistinguible de no tener ninguna. La paradoja de la elección aplicada al quant trading: mas opciones no significa mejores decisiones, significa mas tiempo decidiendo cual usar. V1 necesita exactamente los 3 metodos que sabemos que funcionan.

2. Deteccion de regimen de 3 capas. Volatilidad + tendencia + liquidez es el modelo completo que usan los fondos cuantitativos institucionales. Pero un sistema que no ha cerrado su feedback loop no necesita un modelo institucional de regimen. Necesita saber si el mercado esta tranquilo o agitado. Esa informacion la da la volatilidad sola. Las otras dos capas se anaden cuando haya datos para calibrarlas y el SDE demuestre que la primera capa produce valor.

3. Block Bootstrap en Monte Carlo para V1. El Fisher-Yates shuffle actual es estadisticamente inferior al Block Bootstrap porque destruye la estructura temporal de los trades. Pero la pregunta relevante no es "cual es estadisticamente superior?" sino "la diferencia afecta las decisiones del SDE?". Si el shuffle y el block bootstrap producen el mismo verdict (TRADE vs REJECT), la mejora es academica, no operativa. Se implementa en V2 si el SDE muestra que los resultados del shuffle son inconsistentes con el rendimiento real.

8.5 Veredicto

VEREDICTO: 5 componentes ELIMINADOS, 11 RETRASADOS, 8 SIMPLIFICADOS. La regla es: si no cierra el feedback loop o no produce una decisión accionable, no se construye en V1. Sobreingeniería = complejidad sin retorno operativo. Cada línea de código que no contribuye directamente a producir decisiones correctas es deuda técnica.

9. Arquitectura Final Consolidada

9.1 Pipeline completo

El pipeline de datos del sistema, desde la señal hasta la ejecución, queda definido de la siguiente manera:

Et ap a	Componente	Input	Output	Estado
---------------	------------	-------	--------	--------

1	Backtest Engine	TradingSystem + OHLCV	BacktestResult + metricas	EXISTE
2	Monte Carlo Simulator	Trades PnL + config	MCResult (RoR, P95 DD, prob. profit)	EXISTE
3	Walk-Forward Engine	TradingSystem + OHLCV	WFResult (WFE, stability, robustness)	EXISTE
4	Operability Score	Token + fees + slippage	Score 0-100 + viabilidad	EXISTE
5	Strategy Decision Engine	Backtest + MC + WF + Operability	SDEDecision (action + confidence + capital)	NUEVO
6	Capital Allocation	SDEDecision + portfolio state	Position size + method	CONECTAR
7	Paper Trading	Signals + position size	Trades + PnL + track record	EXISTE
8	Decision Audit	SDEDecision + actual results	predictionCorrect + reconciliation	NUEVO
9	Kill Switches	Portfolio DD + strategy DD	Auto-pause + alerts	NUEVO
10	Execution Layer	Validated signals + real wallet	On-chain transactions	V3

Tabla 8. Pipeline completo del sistema

9.2 SDE: Logica de decision completa

El Strategy Decision Engine ejecuta el siguiente algoritmo para cada estrategia evaluada:

```
function evaluateStrategy(strategyId, riskProfile): // CAPA 1: VETOS ABSOLUTOS
backtest = loadBacktestResults(strategyId) mc = runMonteCarlo(strategyId) wf =
runWalkForward(strategyId) if mc.riskOfRuin > ROR_THRESHOLD[riskProfile]: return
REJECT if backtest.totalTrades < MIN_TRADES[riskProfile]: return REJECT if
backtest.maxDrawdownPct > MAX_DD[riskProfile]: return REJECT if wf.aggregateWFE <
MIN_WFE[riskProfile]: return REJECT if backtest.winRate < 0.35 AND
backtest.payoffRatio < 2.5: return REJECT // CAPA 2: CONFIDENCE-WEIGHTED SCORING
scores = [ { id: "mc", score: mc.compositeScore, confidence: mc.simCount / 10000
}, { id: "wf", score: wf.robustnessScore, confidence: wf.windowCount / 8 }, { id:
"bt", score: backtest.sharpeNormalized, confidence: backtest.dataYears / 2 }, {
id: "oper", score: operabilityScore, confidence: dataQualityScore / 100 }, ]
composite = SUM(score * confidence) / SUM(confidence) // CAPA 3: ACTION
DETERMINATION if composite >= 70: action = TRADE else if composite >= 50: action
= WATCH // paper trading else if composite >= 40: action = HOLD // mantener, no
aumentar else if composite >= 25: action = RETRAIN else: action = REJECT // CAPA
4: CAPITAL RECOMMENDATION if action == TRADE: method =
selectAllocationMethod(regime, activeStrategies) targetPct =
calculateAllocation(method, composite, portfolio) // CAPA 5: AUDIT REGISTRATION
```

```
logDecision(strategyId, action, composite, predictedReturn, predictedDD) return {  
  action, confidence: composite, capitalAction, method, targetPct }
```

9.3 Modelo de datos consolidado

Los cambios al schema de Prisma necesarios para soportar la arquitectura final son:

Modelo	Campos nuevos	Proposito
DecisionLog	predictedAction, predictedConfidence, predictedReturnPct, predictedMaxDD, actualReturnPct, actualMaxDD, actualOutcome, predictionCorrect, reconciledAt, reconciliationDays	Decision Audit Engine
DecisionLog	mcRiskOfRuin, wfe, overfittingScore, robustnessScore, stabilityScore, vetoResults (JSON), moduleContributions (JSON)	SDE output tracking
TradingSystem	sdeAction, sdeConfidence, sdeLastReview, sdeNextReview	Estado del SDE por estrategia
RiskBudget (NUEVO)	maxPortfolioDD, maxStrategyDD, maxDailyVaR, maxConcentrationPct, maxSectorPct, maxChainPct	Kill switches + allocation limits
PortfolioSnapshot (NUEVO)	totalValue, totalDD, activeStrategies, totalExposurePct, equityCurve (JSON), calculatedAt	Portfolio-level tracking

Tabla 9. Cambios al schema de Prisma

10. Roadmap Revisado

El roadmap original tenia 6 sprints en 17 semanas. El roadmap revisado tiene 4 sprints en 8-10 semanas. La reduccion viene de eliminar sobreingenieria, retrasar funcionalidades no esenciales, y priorizar el cierre del feedback loop sobre la expansion de capacidades.

10.1 Sprint 0: Foundation Fixes [3-4 dias]

ID	Tarea	Detalle	Esfuerzo
S0.1	Evolution PRNG	Math.random() a seeded LCG en mutateParams() (14 llamadas)	2h
S0.2	Capital Allocation a Paper Trading	Reemplazar calculatePositionSize() con CapitalAllocationEngine (3 metodos)	3h

S0.3	PAUSED bloquea Paper Trading	Si strategy.status=PAUSED, skip en paper trading scan	1h
S0.4	Bug B5	saveControlsMutation setTimeout(500) a proper mutation	1h
S0.5	Bug B6	smart-money-sync/route.ts import incorrecto	30min
S0.6	Bug B7	phase-strategy-engine.ts token type incompleto	1h
S0.7	Bug B8/B9	strategy-marketplace.tsx CATEGORY_META missing bg	1h
S0.8	Renombrar decision-engine	decision-engine.ts a token-decision-engine.ts	30min

Tabla 10. Sprint 0: Foundation Fixes

10.2 Sprint 1: Strategy Decision Engine [2-3 semanas]

Este es el sprint que transforma la plataforma de herramienta de analisis a sistema de trading.

ID	Tarea	Prioridad
S1.1	Crear strategy-decision-engine.ts con pipeline: vetos a scores a action a capital	P0
S1.2	Extender DecisionLog con campos de SDE + prediccion (audit engine)	P0
S1.3	Implementar Confidence-Weighted Scoring (sin pesos fijos)	P0
S1.4	API: /api/strategy-decision/validate	P0
S1.5	API: /api/strategy-decision/portfolio-review	P0
S1.6	Conectar MC output al SDE (RoR a veto, P95 DD a allocation)	P0
S1.7	Conectar WF output al SDE (WFE a robustness, stability a stability)	P0
S1.8	Reconciliation job diario (Decision Audit Engine)	P0
S1.9	API: /api/strategy-decision/status (dashboard de estado)	P1
S1.10	UI: Decision Dashboard (tabla con action, confidence, capital)	P1

Tabla 11. Sprint 1: Strategy Decision Engine

Primer milestone: Ejecutar /api/strategy-decision/validate sobre una estrategia existente y obtener un SDEDecision con action, confidence, vetoResults, moduleContributions, y capitalAction.

10.3 Sprint 2: Kill Switches + Risk Budget [1-2 semanas]

ID	Tarea	Prioridad
S2.1	Prisma: crear modelo RiskBudget + PortfolioSnapshot	P0
S2.2	Portfolio Kill Switch: auto-pause si portfolio DD > 20%	P0
S2.3	Strategy Kill Switch: auto-pause si estrategia DD > 30%	P0
S2.4	Position Emergency Close: auto-close si posicion pierde > 50%	P0
S2.5	Manual Kill Switch UI: boton global pause + per-strategy pause	P0
S2.6	Concentration Limits: token 15%, sector 30%, chain 50%	P0
S2.7	Proactive Alerts: "Approaching kill switch at X%"	P1
S2.8	Evolution Tree a SDE: promocion de candidatos con umbral 15%	P1

Tabla 12. Sprint 2: Kill Switches + Risk Budget

10.4 Sprint 3: Portfolio Monitor + Volatility Regime [1-2 semanas]

ID	Tarea	Prioridad
S3.1	Volatility Regime Detector (1 capa: rolling vol percentil)	P0
S3.2	Conectar regime al SDE: vol > P75 a Volatility Targeting	P0
S3.3	Dynamic allocation method selection por regimen + action	P0
S3.4	Portfolio Dashboard: equity curve, DD, Sharpe, allocation breakdown	P1
S3.5	Strategy status en Paper Trading UI (action tag + confidence)	P1

Tabla 13. Sprint 3: Portfolio Monitor + Volatility Regime

Milestone de validacion del sistema completo: Al finalizar Sprint 3, el sistema debe poder: (1) evaluar una estrategia con SDE y producir un action tag, (2) asignar capital segun el action tag y el metodo de allocation seleccionado, (3) pausar automaticamente si los kill switches se activan, (4) auditar sus propias predicciones contra resultados reales, y (5) ajustar el metodo de allocation segun regimen de volatilidad.

10.5 Sprint 4+ (V2): Mejoras institucionales [post-validacion]

Estos sprints solo se ejecutan DESPUES de que el sistema complete Sprint 3 y demuestre en paper trading que el SDE produce decisiones correctas (tasa de acierto > 55% en 50+ decisiones reconciliadas). Sin esta validacion, las mejoras institucionales son sobreingenieria prematura.

Sprint 4 (V2): MC Block Bootstrap + Stress Scenarios, WF Parameter Drift, Regime Detection 3 capas, Markowitz, Adaptive position sizing.

Sprint 5 (V3): Execution Layer (Jupiter/Solana), Wallet management, Execution quality metrics, Fee accounting real.

11. Decisiones de Diseño Fundamentales

Este documento consolida y actualiza las decisiones de diseño fundamentales del proyecto. Estas son las reglas de oro que guían toda implementación:

1. Vetos antes que scores. Un veto duro SIEMPRE tiene prioridad sobre un score alto. Una estrategia con Sharpe 3.0 pero Risk of Ruin 15% es REJECT, no TRADE. Los vetos son binarios y no negociables.

2. Acción antes que información. Cada output del sistema debe ser directamente accionable. "La estrategia tiene WFE 45%" es información. "RETRAIN la estrategia" es una decisión. El SDE produce decisiones, no informes.

3. Portfolio antes que estrategia. Las decisiones se toman a nivel de portfolio. Una estrategia individual puede ser excelente, pero si está correlacionada con otras posiciones, el riesgo portfolio-level puede ser inaceptable. Los kill switches operan a nivel de portfolio primero.

4. Feedback loop antes que optimización. Cerrar el feedback loop (Decision Audit Engine + kill switches) es más importante que mejorar cualquier módulo individual. Un sistema que aprende de sus errores supera a uno que tiene mejores inputs pero no puede corregirse.

5. Conservador por defecto. Cuando hay incertidumbre (datos insuficientes, régimen poco claro, MC percentiles amplios), el sistema debe reducir exposición, no aumentarla. El default es Half-Kelly (no full Kelly), y Volatility Targeting solo reduce, nunca aumenta allocation.

6. Reproducibilidad obligatoria. Toda simulación debe ser reproducible con su seed. Esto aplica a MC, Walk-Forward, y Evolution. Math.random() está prohibido en cualquier módulo que produzca outputs que alimenten decisiones.

7. Transparencia radical. Cada decisión del SDE viene con su razonamiento completo: que vetos se evaluaron, que scores se calcularon, que confianza tuvo cada módulo, y que peso efectivo tuvo en la decisión final. El usuario puede auditar cualquier decisión.

8. Simplicidad antes que cobertura. 3 métodos de allocation bien conectados son superiores a 16 métodos desconectados. 1 capa de régimen que funcione es mejor que 3 capas que nadie calibra. La complejidad sin retorno operativo es deuda técnica.

9. Validación antes que construcción. No se añade funcionalidad nueva hasta que la funcionalidad existente esté validada. El sistema demuestra que produce decisiones correctas ANTES de expandir

capacidades. El milestone de validacion (50+ decisiones reconciliadas con > 55% acierto) es obligatorio antes de Sprint 4.

10. Migracion gradual, nunca swap. Cuando una estrategia hija reemplaza a la madre, la migracion de capital es gradual (25%/50%/100%) con verificacion del SDE en cada paso. Nunca se transfiere todo el capital de una vez. Esta regla aplica a todos los cambios de allocation que afecten mas del 30% del portfolio.